Triangles, Step by Step

Gabriel Campbell

May 12, 2018
Last updated September 22, 2018.

The goal of this document is to give the reader an understanding of the core of computer graphics, by explaining step-by-step how to draw perfect triangles. I aim to avoid the family of papers littered with jargon which dominate the Google search results for the specifics of rasterization and to compile as much as possible of that information into a tight and convenient space, as it seemed to me (as a self-taught high school student) near impossible to learn and comprehend some of the specifics required in building a software renderer. I assume that the reader is familiar with programming (I will be using C) and bitwise operations. While a basic understanding of computer graphics would be beneficial, this document assumes little prior knowledge of computer graphics and its conventions.
The examples in this document also assume little about the handedness of the coordinate system and can easily be adapted to other styles.
I have spent the last five months working on implementing a personal-use software graphics library, because of the lack of comprehensive learning material and because I want to make sure that any projects I pour my heart into are as secure and portable as possible. One of the advantages of writing your own graphics libraries, if not to become a better graphics programmer, is that you don't have to lean on the graphics card for the quality or existence of the required drivers. It's also, surprisingly, really fun.

Table of Contents

Display Arrays
Basics of RGBA
Pixel Encoding and Decoding
Rasterization
A Naive Triangle Rasterizer
Barycentric Coordinates
Introduction to Fixed-Point
Fixed-Point Implementation
Fill Rules
The W Coordinate
Perspective Correction
Perspective Division
Normalized Device Coordinates
Clipping and Clip Space
Clipping Lines
Clipping Triangles
Near Plane, Far Plane, and Depth Conversion
Review of the Pipeline
Depth-buffering
Optimizations
Conclusion

Display Arrays

The display is really a simple concept; it is a one-dimensional array of pixel values with a resolution of width by height. Pixel coordinates (0,0) refer to the top left of the display; this has a pixel index of 0. To calculate a pixel's index, we use the following formula:

$$x + (y * width)$$

Where *x* is the pixel X coordinate, *y* is the pixel Y coordinate, and *width* is the width, in pixels, of the display.

So, X coordinate of 0 is at the left edge, X coordinate of *width*-1 is at the right edge, Y coordinate of 0 is at the top edge, and Y coordinate of *height*-1 is at the bottom edge.
The pixel count (element count) of a display is given by *width***height*, and, unless the display is compressed, multiplied by the number of color components (e.g. RGB has 3 color components). Display compression will be discussed in *Pixel Encoding and Decoding*.

Basics of RGBA
In computer graphics, we usually give red (R), green (G), blue (B), and alpha (A) values in the range `[0,1]`. This makes it easy for us humans to read, manipulate, and mix them. This range of values is also independent of the target display's color depth. 1 is full intensity, 0 is the opposite; and alpha is generally used to represent opaqueness (i.e., 1 is fully opaque).
To get the actual R, G, B, or A to be plotted on the display given the normalized color value, multiply the normalized color component value by the maximum value of that color component; or divide to perform the reverse. The maximum of a color component varies based on that component's color depth (color depth can refer to either the number of bits per pixel or per color component of a pixel, which is interpreted as an unsigned integer).

Given that we know the color depth of a component, the component's maximum value is given by:

$$x^2 - 1$$

Where *x* is the number of bits assigned to the component.

Most modern displays use 32 bits (4 bytes) per-pixel, with 8 bits assigned to each component (giving the components a range of `[0,255]`, where 255 is full intensity).

Pixel Encoding and Decoding
Some image arrays may be compressed and some uncompressed, and there are some specific advantages to each. One such advantage is that, for uncompressed 32 bit RGB arrays, you can use the 8 bit unsigned integer type in an array of the format {R,G,B,R,G,B,...} and waste no memory on allocating an alpha channel, while with a compressed format you might not have a 24 bit unsigned integer and be forced to use 32 bits for each pixel, anyway.

Pixel encoding/decoding is a method which allows you to pack more than one pixel component values into a single integer. Take, for example, pixel format R5G6B5; a 16 bit pixel format, which would look like the following when the unsigned integer (pixel value) is viewed in binary format:

```
RRRRRGGGGGGBBBBB
```

This is format will be more efficient than 32 bit, but less accurate.

You can use bitwise operations in order to read or write to specific portions of the pixel values.

In C, pixel encoding and decoding functions should be defined as macro functions, such as:

```
/* encode */
#define RGBA32(R,G,B,A) (uint32_t)((R<<24) | (G<<16) | (B<<8) | A)
/* decode */
#define RGBA32_R(x) (uint8_t)((x & 0xFF000000)>>24)
...
```

Rasterization

At the core of any renderer lies a few of the many possible methods of rasterization. A rasterizer takes a set of vertices and determines (preferably as fast as possible), the pixel coordinates that lie between them. You can visualize and tinker with rasterization as a black and white screen where the white represent the pixels of rastered shapes and the black represents area not yet covered by the rasterizer. Rastered primitives are usually only triangles, lines, and points; anything else, such as quads, can be (and should be, as triangle rasterization is fast) tessellated into triangles. In addition, a rasterizer will almost always calculate factors for interpolation of values for each pixel of a rastered primitive. More on this later.

A Naive Triangle Rasterizer

So, without further ado, here is a basic triangle rasterization algorithm. I will be using a scanline algorithm which starts at the top vertex and goes to the bottom vertex, adjusting the scanline beginning and end X coordinates to effectively trace the outline of a rastered triangle. Note that our rasterizer is supplied the vertex positions in pixel coordinates.

```
struct vec2 { float x, y; };

/* we will raster each triangle in halves (i.e., horizontally split from the
vertically middle vertex); as we can then use simple slope increments to
advance down Y. We must also ensure v0.y <= v1.y <= v2.y due to the way our
algorithm is set up. */
void raster_triangle_half( vec2 v0, vec2 v1, vec2 v2 );

void draw_triangle( vec2 v0, vec2 v1, vec2 v2 ) {
	vec2 tmp;

	if(v2.y < v1.y)
		tmp=v2, v2=v1, v1=tmp;
	if(v1.y < v0.y)
		tmp=v1, v1=v0, v0=tmp;
	if(v2.y < v1.y)
		tmp=v2, v2=v1, v1=tmp;
	if(v1.y == v2.y
	|| v0.y == v1.y)
		raster_triangle_half(v0, v1, v2);
	else {
		vec2 v;
```

```
            v.x = v0.x + ((v1.y-v0.y) / (v2.y-v0.y)) * (v2.x-v0.x);
            v.y = v1.y;
            raster_triangle_half(v0, v1, v);
            raster_triangle_half(v1, v, v2);
      }
}

void raster_triangle_half( vec2 v0, vec2 v1, vec2 v2 ) {
      if(v0.y == v1.y) { /* flat top */
            float invslope1 = (v2.x - v0.x) / (v2.y - v0.y);
            float invslope2 = (v2.x - v1.x) / (v2.y - v1.y);
            float cfx1 = v2.x;
            float cfx2 = v2.x;
            for(int32_t sy = v2.y; sy > v0.y; sy--) {
                  int32_t lx, rx;    /* left and right X */
                  if(cfx1 <= cfx2) lx = cfx1, rx = cfx2;
                  else lx = cfx2, rx = cfx1;
                  /* fill line between (lx,sy) and (rx,sy) */
                  cfx1 -= invslope1;
                  cfx2 -= invslope2;
            }
      }

      if(v1.y == v2.y) { /* flat bottom */
            float invslope1 = (v1.x - v0.x) / (v1.y - v0.y);
            float invslope2 = (v2.x - v0.x) / (v2.y - v0.y);
            float cfx1 = v0.x;
            float cfx2 = v0.x;
            for(int32_t sy = v0.y; sy < v1.y; sy++) {
                  int32_t lx, rx;    /* left and right X */
                  if(cfx1 <= cfx2) lx = cfx1, rx = cfx2;
                  else lx = cfx2, rx = cfx1;
                  /* fill line between (lx,sy) and (rx,sy) */
                  cfx1 += invslope1;
                  cfx2 += invslope2;
            }
      }
}
```

Keep in mind that this is a very, very naive implementation of a rasterizer and should not be used. This will be improved throughout the document to a much more usable form.

Barycentric Coordinates

Used for interpolating values across primitives, a pair of barycentric coordinates tell you the percent of each vertex's attributes each rastered pixel has. We say that an (x,y,z) pair of barycentric coordinates for a point in a triangle has *x* as the percent of the first vertex, *y* as the percent of the second vertex, and *z* as the percent of the third vertex. On a line, we could say that *x* is the percent of the first vertex and *y* percent of the second vertex, and *z* could simply go unused. There are several ways to calculate the barycentric coordinates of a point on a triangle, and some are more efficient than others.

```
/* let bc = barycentric coords, v0 = pixel coords of 1st vertex, v1 = pixel
coords of 2nd vertex, v2 = pixel coords of 3rd vertex, and pt = pixel coords
of point to sample from */

/* this is the most efficient solution when working in 2D */
vec3 bc;
vec3 a = v1 - v0;
vec3 b = v2 - v0;
vec3 c = pt - v0;
float denom = 1.0/(a.x*b.y-b.x*a.y);
bc.y = (c.x*b.y-b.x*c.y)*denom;
bc.z = (a.x*c.y-c.x*a.y)*denom;
bc.x = 1.0 - bc.y - bc.z;
```

Note that *a*, *b*, and *denom* only need to be calculated once per-triangle.

Another (horribly inefficient, but the most simple) way to calculate barycentric coordinates for a given point is to calculate the area of the triangle portion between the opposite vertices and the point, divided by the total area of the triangle.

If the sum of the barycentric coordinates is not equal to 1, it means the point sampled is outside the triangle edges. Our final rasterizer will not sample invalid barycentric coordinates (as all rastered pixel centers will be inside the triangle edges), however our naive rasterizer might. This is problematic for many reasons, most notably that, when texturing, interpolation of bad texture coordinates could easily result in a segmentation fault when we try to access from outside of the range of the texture data.

Given barycentric coordinates, *bc*, of a point in a triangle, we can get the interpolated value for the point using:

$$V = x * bc_x + y * bc_y + z * bc_z$$

Where V is the interpolated value, bc_x, bc_y, and bc_z refer to their appropriate barycentric coordinate, and x, y, and z are the values at the first, second, and third vertex, respectively.

Now I will present our naive rasterizer once again, but this time, I will implement barycentric coordinates. Because we split the triangle in half before rasterization, we will create a global copy of the original vertices and use that as the interpolation area.

```
struct vec2 { float x, y; };
struct vec3 { float x, y, z; };

vec2 og_v0, og_v1, og_v2;

void raster_triangle_half( vec2 v0, vec2 v1, vec2 v2 );

void draw_triangle( vec2 v0, vec2 v1, vec2 v2 ) {
      og_v0 = v0;
      og_v1 = v1;
```

```
	og_v2 = v2;
	vec2 tmp;
	if(v2.y < v1.y)
		tmp=v2, v2=v1, v1=tmp;
	if(v1.y < v0.y)
		tmp=v1, v1=v0, v0=tmp;
	if(v2.y < v1.y)
		tmp=v2, v2=v1, v1=tmp;
	if(v1.y == v2.y
	|| v0.y == v1.y)
		raster_triangle_half(v0, v1, v2);
	else {
		vec2 v;
		v.x = v0.x + ((v1.y-v0.y) / (v2.y-v0.y)) * (v2.x-v0.x);
		v.y = v1.y;
		raster_triangle_half(v0, v1, v);
		raster_triangle_half(v1, v, v2);
	}
}

vec2 sub_vec2( vec2 a, vec2 b ) {
	vec2 v = { a.x - b.x, a.y - b.y };
	return v;
}

void raster_triangle_half( vec2 v0, vec2 v1, vec2 v2 ) {
	/* cacheable */
	vec2 a = sub_vec2(og_v1, og_v0);
	vec2 b = sub_vec2(og_v2, og_v0);
	float denom = 1.0f/(a.x*b.y-b.x*a.y);
	if(v0.y == v1.y) { /* flat top */
		float invslope1 = (v2.x - v0.x) / (v2.y - v0.y);
		float invslope2 = (v2.x - v1.x) / (v2.y - v1.y);
		float cfx1 = v2.x;
		float cfx2 = v2.x;
		for(int32_t sy = v2.y; sy > v0.y; sy--) {
			int32_t lx, rx;	/* left and right X */
			if(cfx1 <= cfx2) lx = cfx1, rx = cfx2;
			else lx = cfx2, rx = cfx1;
			/* fill line between (lx,sy) and (rx,sy) */
			for(int32_t x = lx; x <= rx; x++) {
				vec2 c = { x - og_v0.x, sy - og_v0.y };
				vec3 bc;	/* barycentric coordinates of pt */
				bc.y = (c.x*b.y-b.x*c.y)*denom;
				bc.z = (a.x*c.y-c.x*a.y)*denom;
				bc.x = 1.0f - bc.y - bc.z;
				if(bc.x + bc.y + bc.z == 1.0f)
					/* raster pixel (x,sy) */
			}
			cfx1 -= invslope1;
			cfx2 -= invslope2;
```

```
		}
	}

	if(v1.y == v2.y) { /* flat bottom */
		float invslope1 = (v1.x - v0.x) / (v1.y - v0.y);
		float invslope2 = (v2.x - v0.x) / (v2.y - v0.y);
		float cfx1 = v0.x;
		float cfx2 = v0.x;
		for(int32_t sy = v0.y; sy < v1.y; sy++) {
			int32_t lx, rx;	/* left and right X */
			if(cfx1 <= cfx2) lx = cfx1, rx = cfx2;
			else lx = cfx2, rx = cfx1;
			/* fill line between (lx,sy) and (rx,sy) */
			for(int32_t x = lx; x <= rx; x++) {
				vec2 c = { x - og_v0.x, sy - og_v0.y };
				vec3 bc;	/* barycentric coordinates of pt */
				bc.y = (c.x*b.y-b.x*c.y)*denom;
				bc.z = (a.x*c.y-c.x*a.y)*denom;
				bc.x = 1.0f - bc.y - bc.z;
				if(bc.x + bc.y + bc.z == 1.0f)
					/* raster pixel (x,sy) */
			}
			cfx1 += invslope1;
			cfx2 += invslope2;
		}
	}
}
```

Introduction to Fixed-Point

A large problem with our current rasterizer is that it does not use fixed-point. Hardware rasterizers (such as OpenGL, Direct3D, etc.) implement sub-pixel precision (fractional pixel coordinates) by using fixed-point, because floating-point operations far too often experience random precision loss; also, as they are integer operations, fixed-point is faster. An advantage of fixed-point is that you have a consistent precision, and, given enough fractional bits, precision loss is negligible.

First, we will go over the basics of fixed-point arithmetic.

Fixed-point integers may, following the rules presented here, be signed or unsigned.

Fixed-point notation:

$m.n$

Where m is the number of whole bits, and n is the number of fractional bits.

For example, 16.16 fixed-point is a 32 bit (signed or unsigned) integer with the first 16 bits representing the whole part, and the last 16 representing the fractional part.

The scaling factor (multiply to convert from floating-point to fixed-point, or divide to convert from fixed-point to floating-point) is:

2^n

The smallest fractional number (and the fixed-point to floating-point multiplicand) we can represent is given by:
$1/(2^n)$

Note that we should be aware of cases such as overflow and underflow, and use unsigned integers when negative values are not required.

To multiply two fixed-point numbers, cast first to an integer (of the same signedness) with a bit count which is greater than or equal to the summed fractional bits of both, summed with the summed whole bits of both; that is, after multiplication of fixed-point numbers 24.8 * 16.16, you may end up with a fixed-point up to 40.24, so you should cast the result to a 64 bit signed or unsigned integer.

Assuming we are dividing two fixed-point numbers of the same resolution, we could simply perform floating-point division as-is and then multiply by the scaling factor to get back to fixed-point.

Addition and subtraction can easily be performed on two fixed-points of the same resolution, with a result of the same resolution.

There are many details about and different ways to perform fixed-point arithmetic, but this is what you will need to know. Also, a trick we will make heavy use of:
Bit-shifting (a highly efficient operation) an integer (signed or unsigned) left by x bits multiplies it by x^2, and bit-shifting it right divides it by x^2. Whole number x to 24.8 fixed-point is a simple operation: $x<<8$, and retrieving the whole number of a 24.8 fixed-point is another simple operation: $x>>8$.

Fixed-Point Implementation
Our fixed-point rasterizer will use 24.8 fixed-point for sub-pixel precision (meaning it is on par with OpenGL and Direct3D). We are using signed integers for pixel coordinates, meaning we can represent pixels with center X or Y roughly in the range `[-8388607, 8488607]` ($[2^{24}-1]/2=8388607.5$), which is more than enough (and, really, this won't be a limitation once we implement clipping).

```
struct vec2   { float x, y; };
struct vec2i  { int32_t x, y; };
struct vec2ui { uint32_t x, y; };
struct vec3   { float x, y, z; };
struct vec3ui { uint32_t x, y, z; };

vec2i og_v0, og_v1, og_v2;     /* 24.8 original coordinates */

void raster_triangle_half( vec2 v0, vec2 v1, vec2 v2 );

void draw_triangle( vec2 v0, vec2 v1, vec2 v2 ) {
      og_v0 = { v0.x*256.0f, v0.y*256.0f };
```

```
	og_v1 = { v1.x*256.0f, v1.y*256.0f };
	og_v2 = { v2.x*256.0f, v2.y*256.0f };
	vec2 tmp;
	if(v2.y < v1.y)
		tmp=v2, v2=v1, v1=tmp;
	if(v1.y < v0.y)
		tmp=v1, v1=v0, v0=tmp;
	if(v2.y < v1.y)
		tmp=v2, v2=v1, v1=tmp;
	if(v1.y == v2.y
	|| v0.y == v1.y)
		raster_triangle_half(v0, v1, v2);
	else {
		vec2 v;
		v.x = v0.x+((v1.y-v0.y)/(v2.y-v0.y))*(v2.x-v0.x);
		v.y = v1.y;
		raster_triangle_half(v0, v1, v);
		raster_triangle_half(v1, v, v2);
	}
}

vec2i sub_vec2i( vec2i a, vec2i b ) {
	vec2i v = { a.x - b.x, a.y - b.y };
	return v;
}

void raster_triangle_half( vec2 v0, vec2 v1, vec2 v2 ) {
	int32_t x0 = v0.x*256.0f, y0 = v0.y*256.0f;
	int32_t x1 = v1.x*256.0f, y1 = v1.y*256.0f;
	int32_t x2 = v2.x*256.0f, y2 = v2.y*256.0f;
	/* cacheable */
	vec2i a = sub_vec2i(og_v1, og_v0);
	vec2i b = sub_vec2i(og_v2, og_v0);
	float denom = 1.0f/((int64_t)a.x*b.y-(int64_t)b.x*a.y);
	if(v0.y == v1.y) { /* flat top */
		int32_t invslope1 = ((float)(x2-x0) / (float)(y2-y0))*256.0f;
		int32_t invslope2 = ((float)(x2-x1) / (float)(y2-y1))*256.0f;
		int32_t cfx1 = x2;
		int32_t cfx2 = x2;
		for(int32_t sy = y2; sy > y0; sy -= 256) {
			int32_t lx; /* left exact X */
			int32_t rx; /* right exact X */
			if(cfx1 <= cfx2) lx = cfx1, rx = cfx2;
			else lx = cfx2, rx = cfx1;
			/* fill line between (lx,sy) and (rx,sy) */
			for(int32_t x = lx; x < rx; x += 256) {
				vec2i c = { x - og_v0.x, sy - og_v0.y };
				vec3 bc;	/* barycentric coordinates of pt */
				bc.y = ((int64_t)c.x*b.y-(int64_t)b.x*c.y)*denom;
				bc.z = ((int64_t)a.x*c.y-(int64_t)c.x*a.y)*denom;
				bc.x = 1.0f - bc.y - bc.z;
```

```
                        /* we are confident the b-coord pt is inside */
                        /* raster pixel (x>>8,sy>>8) */
                }
                cfx1 -= invslope1;
                cfx2 -= invslope2;
        }
    }

    if(v1.y == v2.y) { /* flat bottom */
        int32_t invslope1 = ((float)(x1-x0) / (float)(y1-y0))*256.0f;
        int32_t invslope2 = ((float)(x2-x0) / (float)(y2-y0))*256.0f;
        int32_t cfx1 = x0;
        int32_t cfx2 = x0;
        for(int32_t sy = y0; sy < y1; sy += 256) {
            int32_t lx; /* left exact X */
            int32_t rx; /* right exact X */
            if(cfx1 <= cfx2) lx = cfx1, rx = cfx2;
            else lx = cfx2, rx = cfx1;
            /* fill line between (lx,sy) and (rx,sy) */
            for(int32_t x = lx; x < rx; x += 256) {
                vec2i c = { x - og_v0.x, sy - og_v0.y };
                vec3 bc;    /* barycentric coordinates of pt */
                bc.y = ((int64_t)c.x*b.y-(int64_t)b.x*c.y)*denom;
                bc.z = ((int64_t)a.x*c.y-(int64_t)c.x*a.y)*denom;
                bc.x = 1.0f - bc.y - bc.z;
                /* we are confident the b-coord pt is inside */
                /* raster pixel (x>>8,sy>>8) */
            }
            cfx1 += invslope1;
            cfx2 += invslope2;
        }
    }
}
```

Note that the `int64` calculations cause horrendous performance when done per-pixel for barycentric calculation. The `int64` cast is done to follow the rules of fixed-point multiplication; a 24.8 fixed-point multiplied by 24.8 could result in a fixed-point up to 48.16. If we omit these casts, we will almost always have overflows which result in garbage which can look quite psychedelic.

Thankfully, the solution is simple; calculate once per rastered triangle half the barycentric coordinates of the vertices, and find both edge's (our inverse slope calculations refer to them as edge 0 and edge 1) barycentric coordinate deltas per Y (change in barycentric coordinates / height of triangle half). Then increment the appropriate values to two copies of the apex's barycentric coordinates as you traverse up or down the triangle half; this way, we know the scanline left/right barycentric coordinates. Then for each row calculate the left-to-right barycentric coordinate differences divided by the row's pixel width, and increment by that as you traverse through the pixels of the scanline. Now, we only need to calculate barycentric coordinates three times per half, one division per row, and the rest of the time, we only need to use a few additions!

Fill Rules
In order to prevent confusion in the overlapping of touching triangle edges, a rasterizer requires a set of rules for how to fill when a pixel's center falls on the edge of primitives; also known as a fill convention. The most common and widely accepted solution is the top-left fill rule, used in graphics APIs such as OpenGL and Direct3D.

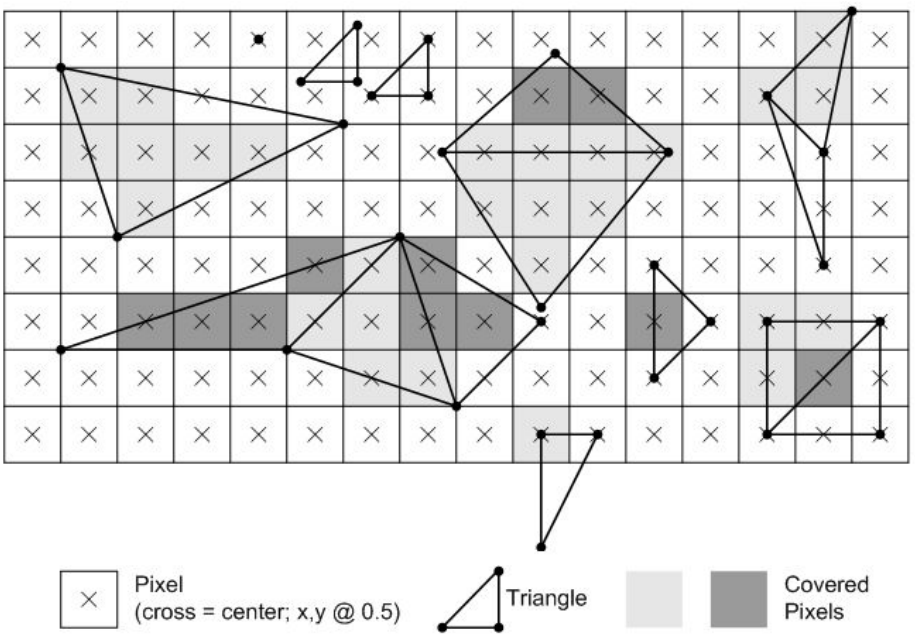
A triangle edge can be thought of as an infinitely thin line between two vertices. Keeping in mind the sub-pixel precision of pixels, the top-left fill rule is as follows:
Raster any pixel with a center that falls inside (or on an edge of) the triangle if pixel center X is < exact right edge, pixel center X >= exact left edge, pixel center Y >= exact top edge, pixel center Y < exact bottom edge, where top and bottom edge is the topmost and bottommost Y of a rastered triangle half and the left and right X are of the current scanline at the exact Y. More precisely, the top-left fill rule defines that the "top edge" only exists if the triangle being rastered has a completely horizontal edge that is above all others, otherwise the pixel is only valid if its center Y > the topmost Y. Therefore, center Y >= topmost Y applies when rasterizing flat-topped triangle halves and center Y > topmost Y otherwise.

Our rasterizer is set up to use the exact Y of the apex of the triangle half and increment/decrement by the size of a pixel (256, in this case), and still retain the exact X from the left endpoint of the scanline, so implementing the top-left fill rule is trivial from here.

We will make the following changes:
To both the halves' scanline rasterization loops:
```
int32_t x_center = (lx>>8)<<8;
if(x_center < lx) lx += 256;
for(int32_t x = lx; (x_center=(x>>8)<<8) < rx; x += 256)
```
And, to the flat top Y loop:
```
int32_t y_center;
for(int32_t sy = y2; (y_center=(sy>>8)<<8) >= y0; sy -= 256)
```
And, finally, to the flat bottom Y loop:
```
int32_t y_center = (y0>>8)<<8;
if(y_center < y0)
      cfx1 += invslope1, cfx2 += invslope2, y0 += 256;
for(int32_t sy = y0; (y_center=(sy>>8)<<8) < y1; sy += 256)
```

We will also need to make all `invslope` variables and their assigned divisions, as well as all `cfx` variables, of type `double`. This is because both 24.8 fixed-point and single-precision floating-point have too much precision loss during interpolation of the edge's X coordinate and leads to gaps between triangle edges. Double-precision floating-point is probably not the most efficient method of solving this problem, but it is the easiest. I suggest tracking the fractional part of the scanline endpoints in an integer and correcting the integer which tracks the whole part once the fractional part overflows.

You can use this illustration from Microsoft's Direct3D rasterization rules article to visualize the top-left fill rule:



https://msdn.microsoft.com/en-us/library/windows/desktop/cc627092(v=vs.85).aspx

The W Coordinate

In 3D computer graphics, we use homogeneous coordinates, which means everything is actually 4D. There are countless uses for the fourth (W) coordinate of a vertex, but generally, it is used to perform projections and to maintain proportions. Assuming you are using OpenGL matrices, a projection matrix multiplied by a vertex results in the final vertex having a W coordinate equal to the world space Z (i.e., the vertex's Z prior to any additional operations), negated. After a vertex has its final 4D position determined, it is considered to be in clip space because it is next used in clipping; this is discussed later in *Clipping*.

The W being world space Z negated is useful, because after a vertex is projected, its Z loses a lot of precision and compatibility with specific uses of the vertex depth, as the Z mapping is extremely unpredictable, while W retains the original Z. The Z coordinate isn't completely useless, as we will see, but generally it is untrustworthy when used in conjunction with perspective. More on this in *Depth-buffering*.

The W coordinate is also useful for perspective division, explained further in a later (aptly named) section.

Another use for the W coordinate is clipping, performed on the clip space vertices of a primitive, where we ensure, for each vertex, `-w <= (x,y,z) <= w`. Clipping will be further discussed in later sections, though note for now that the W is (in OpenGL projection matrices) the negated original Z; perspective division by said W coordinate would then confuse signs of the Z coordinate and cause the vertices that should be behind the camera to be projected in front (which is a nightmare).

The last major W coordinate application to know about is a process called perspective correction (sometimes called perspective correct texture mapping), which uses the fact that W (as a result of perspective projection) is the actual world space depth to make interpolation nonlinear (by modifying the linear barycentric coordinates) and cause it to vary (enlarge/shrink) based on the actual depth of the vertices.

Perspective Correction
To perform perspective correction on the barycentric coordinates of a pixel, we must divide each barycentric coordinate by the related vertex's W, and then divide by the interpolated inverse W of the vertices. It is faster to multiply than divide, so we should cache each vertex's inverse W for each rastered triangle and perform the first pair of divisions that way.
We will do the following series of operations, using the original vertices of the triangle:

Per-triangle:

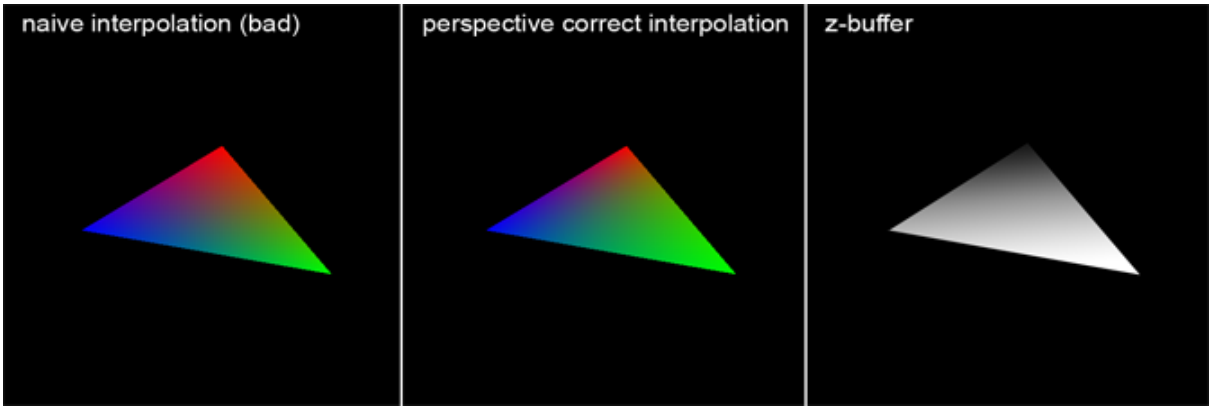
$A = 1/abs(v0_w)$
$B = 1/abs(v1_w)$
$C = 1/abs(v2_w)$

Per-pixel:

$W = 1/(bc_x*A + bc_y*B + bc_z*C)$
$bc_x *= A * W$
$bc_y *= B * W$
$bc_z *= C * W$

Where *abs(x)* is the absolute value.

And, here is an image of perspective correction in action:



https://www.scratchapixel.com/lessons/3d-basic-rendering/rasterization-practical-implementation/perspective-correct-interpolation-vertex-attributes

Fragment and Vertex Shading
Before we continue, I would like to stop to explain a bit about vertex and fragment shading. Vertex and fragment shaders are programs (or simply functions) responsible for determining the final clip space position of a vertex or final fragment color (a fragment is a not yet plotted pixel; but one that lies within a primitive). A vertex shader (run for every vertex to be drawn) might take in the original coordinates of the vertex (considered "object space") and then multiply it by a series of matrices; performing transformations, scalings, rotations, etc., and it will usually multiply it by a projection matrix. It might also calculate and store per-vertex values for later manual interpolation via barycentric coordinates supplied by a fragment shader, but in most cases values "output" by a vertex shader will automatically be interpolated by the renderer before access within the fragment shader. Regardless of what is done, the vertex output from the vertex shader is considered to be in clip space, ready for the pipeline of processing that may ensue. The fragment shader will determine final pixel color

or even discard the fragment. Shaders are not limited to basic operations, though, and fragment and vertex shaders might do hundreds of calculations to perform stuff like lighting. There are also other shaders, such as tessellation shaders, which are used to generate new primitives.

For any software renderer, however, remember; these operations take time. You must approach these things (the fragment shader, especially) strategically. Look around the internet for techniques used in the software renderers of games such as *Thief II: The Dark Project*, *Quake II* and *Unreal Tournament* (1998); some of them are really cool.

Perspective Division
Consider a point on a screen, with a high depth (Z, stored in the W coordinate) coordinate, now divide the (x,y,z) coordinates of the point all by the W; it has the effect of squishing the point towards the origin (0,0,0) (which is, in a 3D renderer, in the center of the screen); and as this W (i.e., original depth) lessens the point gets further and further from the origin as if approaching it; this is a process called perspective division and given a 4D (clip space) point, the 3D point is (`x/w`, `y/w`, `z/w`). Perspective division is performed on each resultant vertex of clipping, a process I will explain in later sections. After perspective division, you end up with normalized device coordinates.

Normalized Device Coordinates
This is a quick explanation of normalized device coordinates (NDC) as seen in OpenGL. Basically, after we've completely run through the pipeline (an overview of which is to be given in *Review of the Pipeline*), we will result in a cube formed by perspective division (and anything that would have been outside of this cube would have been discarded by clipping); and each vertex that was within the viewing frustum (discussed in *Near Plane, Far Plane, and Depth Conversion*) now has x, y, and z in the range `[-1,1]`, thus they are within our NDC cube.

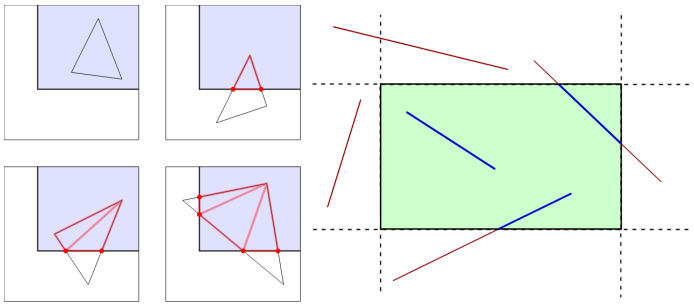
According to OpenGL, NDC can be described by the following:
X of 1 is the right edge of the screen
X of -1 is the left edge of the screen
Y of 1 is the top edge of the screen
Y of -1 is the bottom edge of the screen
Z of 1 is the greatest depth
Z of -1 is the lowest depth

Using the OpenGL coordinate system, and given the origin point of our display is in the top left, the pixel coordinates of an OpenGL NDC point are given by:
$x = w + [v_x * w]$
and
$y = h + [-v_y * h]$
Where x is the pixel coordinate X, y is the pixel coordinate Y, v_x and v_y refer to coordinates of the NDC vertex, and w and h are half the width and half the height of the display.
If you write a renderer and your images look flipped, these equations could be the culprit. This is called the screen transform.

Clipping and Clip Space

Our renderer is almost complete; except now we need to figure out how to perform homogeneous clipping before the perspective division is performed. To ensure that a primitive is within the NDC cube (remember, `[-1,1]` on each axis) the following condition must be met for each vertex of the primitive: `-w <= (x,y,z) <= w`. We can ensure this condition is met by (without affecting the visuals that the programmer expects) discarding or generating new primitives when necessary to clip against the near, far, left, right, top and bottom planes, each defined by the W coordinate. This also has the added benefit of only drawing that which is within the edges of the screen.

Here are some illustrations of our goals:



https://paroj.github.io/gltut/Positioning/Tut05%20Boundaries%20and%20Clipping.html
https://en.wikipedia.org/wiki/Line_clipping

We will define the function

```
bool in_frustum(vec4 v) {
      return (-v.w <= v.x) && (v.x <= v.w)
          && (-v.w <= v.y) && (v.y <= v.w)
          && (-v.w <= v.z) && (v.z <= v.w);
}
```

which will return true if the point is within the frustum and false if it is not (needs to be clipped).

Clipping Lines

The first goal is to implement a method to clip a line; because if we can clip a line, we can also find a way clip triangles.

Below, we will take a look at an efficient line clipping algorithm, adapted from the excellent algorithm found at
https://github.com/hrydgard/ppsspp/blob/master/GPU/Software/Clipper.cpp.
A semi-detailed explanation of a similar algorithm can be found at
https://chaosinmotion.blog/2016/05/22/3d-clipping-in-homogeneous-coordinates/.

```
#define CLIP_POS_X_BIT (1<<0)
#define CLIP_NEG_X_BIT (1<<1)
#define CLIP_POS_Y_BIT (1<<2)
#define CLIP_NEG_Y_BIT (1<<3)
#define CLIP_POS_Z_BIT (1<<4)
#define CLIP_NEG_Z_BIT (1<<5)
/* cohen-sutherland outcode */
uint8_t get_outcode(vec4 v) {
```

```
	uint8_t outcode = 0;
	if(v.x >  v.w) outcode |= CLIP_POS_X_BIT;
	if(v.x < -v.w) outcode |= CLIP_NEG_X_BIT;
	if(v.y >  v.w) outcode |= CLIP_POS_Y_BIT;
	if(v.y < -v.w) outcode |= CLIP_NEG_Y_BIT;
	if(v.z >  v.w) outcode |= CLIP_POS_Z_BIT;
	if(v.z < -v.w) outcode |= CLIP_NEG_Z_BIT;
	return outcode;
}
vec4 lerp(vec4 a, vec4 b, float t) {
	float t1 = 1.0f - t;
	vec4 out;
	out.x = t1 * a.x + t * b.x;
	out.y = t1 * a.y + t * b.y;
	out.z = t1 * a.z + t * b.z;
	out.w = t1 * a.w + t * b.w;
	return out;
}
void clip_line(vec4* a, vec4* b) {
	uint8_t mask_a = get_outcode(*a);
	uint8_t mask_b = get_outcode(*b);
	uint8_t mask = mask_a | mask_b;
	if((mask_a | mask_b) == 0)
		return;	/* line fully visible; trivial accept */
	if(mask_a & mask_b)
		return;	/* line not visible; trivial reject */
	if(mask)
	{
		#define CLIP_LINE(PLANE_BIT, A, B, C, D) \
		{	if(mask & PLANE_BIT) { \
			float dp0=a->x*A + a->y*B + a->z*C + a->w*D; \
			float dp1=b->x*A + b->y*B + b->z*C + b->w*D; \
			if(mask_a & PLANE_BIT && dp0 < 0) {	\
				float t = dp1 / (dp1 - dp0);	\
				*a = lerp(*b, *a, t);		\
			}	\
			dp0 = a->x*A + a->y*B + a->z*C + a->w*D;	\
			if(mask_b & PLANE_BIT && dp1 < 0) {	\
				float t = dp1 / (dp1 - dp0);	\
				*b = lerp(*b, *a, t);		\
			}	\
		}	\
		}
		CLIP_LINE(CLIP_POS_X_BIT, -1, 0, 0, 1);
		CLIP_LINE(CLIP_NEG_X_BIT,  1, 0, 0, 1);
		CLIP_LINE(CLIP_POS_Y_BIT,  0,-1, 0, 1);
		CLIP_LINE(CLIP_NEG_Y_BIT,  0, 1, 0, 1);
		CLIP_LINE(CLIP_POS_Z_BIT,  0, 0,-1, 1);
		CLIP_LINE(CLIP_NEG_Z_BIT,  0, 0, 1, 1);
		#undef CLIP_LINE
	}
```

}

If you do not wish to delve further into how homogeneous clipping works, you should at least note that an outcode (from the Cohen-Sutherland algorithm) sets a “plane bit” when the vertex is on the wrong side of the specific plane. When the two vertices have the same plane bit set (bitwise AND), then, it is a trivial reject scenario (that is, discard the line ENTIRELY) because it means that there is absolutely no way that the line have any portion within the clipping planes. When the bitwise OR of the outcodes two vertices is equal to zero, it means that no plane outcodes were set, so the line can be trivially accepted.
Notice that the A, B, C, D dot product calculation can be simplified for these clipping planes to addition and negation, as we already know the results of all multiplications by 1, -1 or 0.

Clipping Triangles
Triangle clipping, unfortunately, is not as easy as clipping lines. This is because there are many different situations that can be encountered which can require numerous triangle generations. In order to visualize this, draw several 2D bounding boxes on a piece of paper and draw in each a triangle which passes through one or more planes (the sides of the box, in this case), creating a clip point wherever the triangle edge intersects with sides of the box, and then tessellate by connecting points until the portion of the triangle that is inside is made up of triangles which fit within the box.
Evidently, this will become more complicated when the third dimension is considered, but let’s step back and consider how the line clipper works; it clips one line (a triangle is made by three lines) against one plane at a time, for all six planes. If we clip all three lines against one plane at a time, we can quickly see how we can test and retest the clipped point’s outcodes in order to determine when and how to generate triangles. Begin by clipping the triangle against the first plane. Clip all (possibly more than one) result triangles against the next plane, after recalculating the outcode for the vertices of the triangle being clipped. Repeat for all planes. Also, don’t forget to test the initial vertices, prior to any clipping, for trivial accept/reject.
Unfortunately, this is still not a particularly efficient or simple operation, but we will always need to manage several generated triangles and test a fair number of outcodes, no way around it.

Near Plane, Far Plane, and Depth Conversion
The near plane is the plane which marks the closest point to the actual viewpoint, located at Z=-1, that primitives are rendered in; likewise for the far plane, which is the farthest back plane that primitives are rendered in (at Z=1). The near and far plane together form the top and base of a square-based pyramid frustum; called the viewing frustum, and everything within it is drawn. Projection matrices exist for both non-symmetrical and symmetrical viewing frustums. Non-symmetrical frustums allow you to specify the near and far along with the coordinates of the left, right, top, and bottom of the base of the frustum while symmetrical frustums allow you to specify near, far, and the field-of-view and aspect ratio to automatically generate those parameters.

Assuming you are using the coordinates system (and matrices) seen in OpenGL, once a vertex is in NDC space you have the near plane at Z=-1 and the far plane at Z=1. To convert to

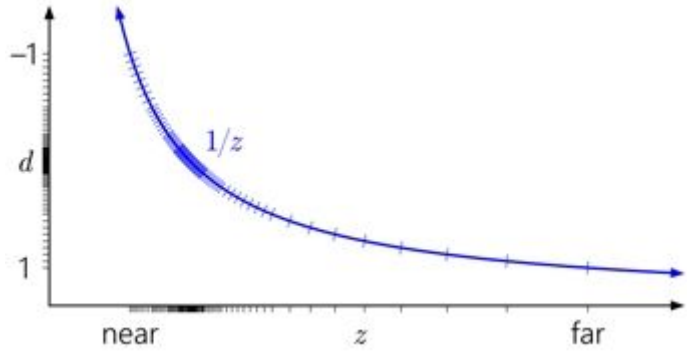
a Z with range `[-1,1]` to a Z with range `[0,1]`, you would do *`z*0.5+0.5`*. To convert from range `[0,1]` to `[-1,1]`, you would do *`z*2-1`*. For later use with depth buffering (discussed in *Depth-buffering*), it is best to convert Z of all of the vertices from range `[-1,1]` to `[0,1]` prior to interpolation. A pixel with a depth in this range can then easily be plotted to a integer-based (fixed-point) depth buffer. If a pixel has a depth outside of this range, however, it is not visible to begin with and should be discarded (though you usually wouldn't get pixels with out-of-range depths if clipping is enabled, anyway).

Review of the Pipeline
To recap, the pipeline run through for each primitive runs along the lines of vertex specification (in object space), to vertex shading (resulting in clip space vertices), to clipping, to perspective division, then from NDC coordinates to screen space, to rasterization.

A vertex is originally in what is conventionally called "object space", usually multiplied by a model matrix (not discussed here, generally consists of translations, rotations, and scaling of the object away from the origin) to end up in "world space", then, once multiplied by a view matrix (not discussed here, but it aligns all of the objects with the camera's position and rotation to emulate camera movement), it is in "view space". The vertex is often then multiplied by a projection matrix. Once the final 4D vertex coordinates are determined, it is considered "clip space". They then go through perspective division and the results are in "NDC", and then go through the screen transform (to end up in "screen space"), which is when the pixel coordinates of a vertex are finally determined.

Depth-buffering
So far, we have only talked about enough to draw the entirety of each triangle in a scene with no regard to which parts may be behind the others. Clearly, this will result in unrealistic scenes. There is a simple solution; for each pixel drawn, we can plot the pixel's depth (Z coordinate) to a second buffer, the depth buffer, which is the size of the display. After performing in the remapping of the NDC Z coordinate from `[-1,1]` to `[0,1]` as discussed in *Near Plane, Far Plane, and Depth Conversion*, 0 is the closest we can draw and 1 is the farthest. Given this, we can say that we should discard any pixel that has a Z greater than or equal to the Z in the depth buffer, as it is obscured. This is called the depth test; if a fragment shader is in use, the depth test is, in most cases, best to be run before it (which is called early depth testing).

Now, this works perfectly; walls and other objects in front of each other will not be drawn. However, there is an issue with this approach, given the way that perspective projections work. Following perspective division, the Z value is in a very strange and sporadic range; often very close to one when in fact the original primitive was nowhere near the far plane. The following diagram provides a visualization of this (note axis *d* is the NDC Z, axis *z* is the view space Z):



https://developer.nvidia.com/content/depth-precision-visualized

This is not an issue when an object is close to the viewer; but causes visual artifacts as objects get further away. It creates an issue called Z-fighting which is when the surface of an object bleeds through another and inconsistently fights for the pixel (which looks horrendous). There are a couple of approaches to solve Z-fighting, such as logarithmic Z-buffers or W-buffers. W-buffering is when you use, in a floating-point depth buffer, the pixel's interpolated W coordinate; this works well because the W coordinate retains, at least according to OpenGL's perspective projection matrix, the view space Z.

Optimizations

In an actual implementation of a 3D software renderer, there are countless opportunities for optimizations. The most vital optimizations for a software renderer are mostly to do with reducing pixel overdraw. Some ideas include:

-OpenCL/SIMD/multithreading acceleration

-Use an adaptive scanline depth buffer which caches the largest scanlines and depth tests the start and end to possibly discard entire scanlines

-Sort vertices of halves by x in initialization to remove the left/right x sorting per-row

-Perspective correct once for every number of pixels along the current scanline, instead of for every pixel

etc.

Conclusion

Throughout this document I have presented the basics of writing a pixel-perfect triangle renderer. I have excluded countless details to make the information within this document as general and concise as possible. I did not cover near half of the details of matrices, vectors, and the thousands of the more common rendering techniques and advanced features such as texture mapping, mipmapping, texture filtering, etc. With many of the issues covered, I have presented only one of many possible solutions. I'll leave it to you to research more about the fundamentals and techniques of computer graphics.